COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	October 2, 2026	**DATE SUBMITTED:**	October 2, 2026
NAME:	Mendoza, Nathaniel B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 10.1 **Graph**

1. PROCEDURE OUTPUT

ILO A

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

Table 10.1 Adjacency List

ILO B.1

```
PS C:\Users\TIPQC\Documents\Eydischt\HOA10p1> cd
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},
```

```
DFS Order of vertices:
1
5
8
6
7
3
4
2
```

Table 10.2 Depth-First Search

ILO B.2

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},
```

```
BFS Order of vertices:
1
2
5
4
8
3
6
7
```

Table 10.3 Breadth-First Search

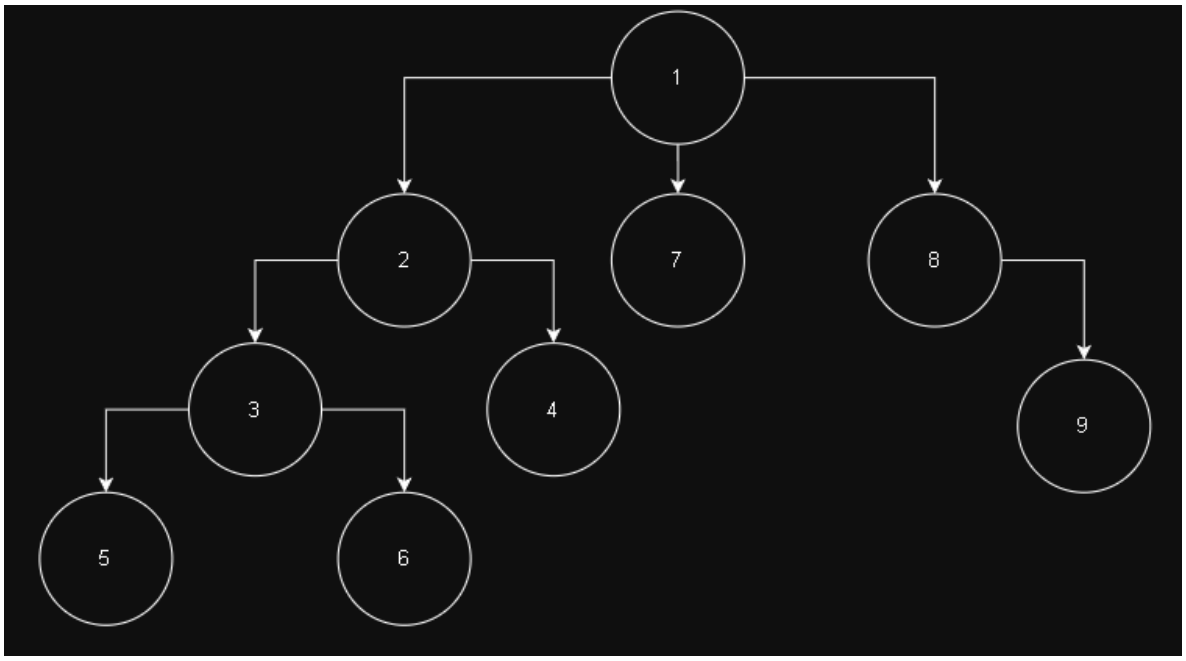
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

The most applicable algorithm here is the DFS or the Depth-First Search as it uses the idea of backtracking where it implements recursions. Which involves exhaustive searches of all the nodes by going ahead. In application the problem, the person wants to visit any location that's indicated within the map where this algorithm does. He starts at a specific node / one location (vertex) then wants to explore every vertex as far as possible then backtracks.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.



Pathing	
Tree preorder: 1 2 3 4 5 6 7 8 9	**Graph DFS (start at 1):** 1 2 3 4 5 6 7 8 9

COMPARISON		
	DFS	**TREE**
Path / Visiting	Visit vertex then goes to unvisited neighbor	After going to node it goes to child
Start	Anywhere you want	Root
Backtracking starts when	No unvisited neighbors are left	No more child
Recursion type	Stack	Stack

PSEUDOCODE COMPARISON

DFS

```
Start
   mark v as visited
   add v to the result list
   for each vertex u adjacent to v do
      if u is not visited then
         GraphDFS(G, u)
      end if
   end for
End
```

Tree

```
Algorithm Preorder(node)
Start
   if node is null then
      return
   end if
   visit(node)
   for each child c of node do
      Preorder(c)
   end for
End
```

CODE

main.cpp > ...

```cpp
#include <iostream>
#include <vector>
using namespace std;

// ---------- Graph DFS (modified from the doc) ----------
void dfsRec(vector<vector<int>> &adj, vector<bool> &visited, int s, vector<int> &res) {
    visited[s] = true;
    res.push_back(s);

    // Recursively visit all adjacent vertices that are not visited yet
    for (int i : adj[s])
        if (visited[i] == false)
            dfsRec(adj, visited, i, res);
}

vector<int> dfs(vector<vector<int>> &adj) {
    vector<bool> visited(adj.size(), false);
    vector<int> res;
    dfsRec(adj, visited, 1, res);          // start at vertex 1
    return res;
}

void addEdge(vector<vector<int>> &adj, int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

// ---------- Tree preorder ----------
struct Node {
    int val;
    vector<Node*> children;
    Node(int v) : val(v) {}
};

void preorder(Node* n) {
    if (!n) return;
    cout << n->val << " ";                 // visit first
    for (Node* c : n->children)            // then go deeper
        preorder(c);
}
```

```cpp
int main() {
    // Graph DFS on vertices 1 to 9
    int V = 10;                                  // index 0 is unused
    vector<vector<int>> adj(V);
    addEdge(adj, 1, 2);
    addEdge(adj, 1, 7);
    addEdge(adj, 1, 8);
    addEdge(adj, 2, 3);
    addEdge(adj, 2, 6);
    addEdge(adj, 3, 4);
    addEdge(adj, 3, 5);
    addEdge(adj, 8, 9);

    vector<int> res = dfs(adj);
    cout << "Graph DFS:      ";
    for (int i = 0; i < res.size(); i++)
        cout << res[i] << " ";
    cout << endl;

    // Tree preorder on the same 9 nodes
    Node* n[10];
    for (int i = 1; i <= 9; i++) n[i] = new Node(i);
    n[1]->children = {n[2], n[7], n[8]};
    n[2]->children = {n[3], n[6]};
    n[3]->children = {n[4], n[5]};
    n[8]->children = {n[9]};

    cout << "Tree preorder:  ";
    preorder(n[1]);
    cout << endl;
    return 0;
}
```

OUTPUT

```
Graph DFS:      1 2 3 4 5 6 7 8 9
Tree preorder:  1 2 3 4 5 6 7 8 9
```

3. In the performed code, what data structure is used to implement the Breadth First Search?

Queue

4. How many times can a node be visited in the BFS?

Exactly once.

CONCLUSION

After performing the activity I learned that with Breadth-First Search (BFS) graph traversal algorithm, each node is visited exactly once. Depth-First Search or the (DFS) on the other hand, uses the idea of backtracking where it implements recursions. Which involves exhaustive searches of all the nodes by going ahead. Lastly, it can be represented as matrix using an adjacency matrix.

REFERENCES

https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.